ROTATE engine: what we built, how, and why

Team notes · 6 Nov 2026 · build steps 1 to 17 · engine: Sudipta

This is the plain-language story of the engine: what each build step did, how, and why it was done that way. It is for teammates who did not build the engine and for anyone answering judges' questions. The exact method and numbers are in ENGINE.md; how to install and call the engine is in INTEGRATION.md. The step-by-step checklist with each step's "Done when" check is Plan/ROTATE engine build steps.md.

What we're trying to do

ROTATE helps a Bangladeshi farmer, or an extension officer sitting with one, compare crop rotations. It doesn't tell anyone what to plant. It puts three options side by side: the rotation the farmer uses now and two others. Each one shows the evidence behind it, the benefits, what we don't know, and the risks to check locally.

The engine is the part that does the reasoning. It takes a site, a decision date and the farmer's choices, and gives back the cards as JSON. The website, map and videos are built by the rest of the team on top of it.

Two ideas sit under it:

- **Look-alike years.** We take the weather in the 60 days before the decision and find the past years whose weather looked most like it. Then we show what the vegetation did in those years. This is not a forecast, and every output says so.
- **A cited crop table.** What a rotation does for water, soil and salt comes from published studies, written down in `rules/rules.json`. NASA data never claims to show how a rotation performed.

Where things stand (6 Nov 2026)

Step	What	Status
0.1–0.6	Questions, accounts, reading, install, practice, team agreements	done
1	Hour-1 checks, AppEEARS NDVI request	done
2	Repo skeleton, first POWER call	done (not committed)
3	Freeze the JSON contract	done
4	Fetch POWER for both sites	done (not committed)
5	Seasons and features	done
6	Look-alike years (analogs)	done
7	Crop rule table, `rules.json`	done as a draft, not yet reviewed
8	Confidence	done (built before 7, agreed)
9	Card rule and cards	done
10	Tests, fixtures, offline check	done

Step	What	Status
11	Bug fixes and the NDVI curve	done
12	Freeze gate and `demo-v1` tag	gate check passed; **tag open** (needs a first commit)
13	Full NDVI: smoothing, per-year curves	done
14	Backtest	done: **no demonstrable skill** at either site
15	Stretch: what-if, tercile, soil wetness, 4th card, extra NDVI	done, all 5 items
16	Final backtest and engine freeze	final run done, same numbers; **`engine-freeze` tag open**
17	Engine doc and integration guide	written and checked on a simulated fresh clone; **a teammate's own run is open**
18	Hand-off	not started

In numbers: 188 tests pass offline in about 16 seconds; 2 cached sites; 14 fixtures; 6 JSON schemas; engine `0.1.0`, contract `1.0`.

The decisions that shape everything

If someone asks "why is it like this?", the answer is usually one of these.

Decision	Why
Match the 60 days *before* the decision, never forecast	It is honest about what the data can do. Past seasons are shown as examples, not as a prediction.
Look-alike years only from Boro 1999 on	POWER's rain, humidity and max temperature jump in 1997 at both sites, so earlier years aren't comparable.
Anwara matches without rain	Its POWER rain is 2–4 times too high from 2015 on (single days up to 663 mm). Using it would match on a data error.
Mahalanobis distance with Ledoit-Wolf covariance, plus a Euclidean check	It stays stable with only about 25 years, and the side-by-side Euclidean run shows whether the choice matters. Never MinCovDet, which down-weights the extreme years we want to find.
Every fit uses training years only	The year being judged never helps judge itself, so nothing leaks.
k = 5, fixed in advance	Fixing it before seeing results means we can't tune it to look good.
Confidence always shows its chance rate	"4 of 5 agree" happens 38% of the time with coin flips. Saying so stops it sounding stronger than it is.
Backtest verdict rule written before the first run	So the verdict can't be bent to fit the numbers. It came out "no demonstrable skill", and that is what every output says.
Crop facts only from cited sources; unknown is `null`	No invented numbers. A `null` shows as "no data", never as zero.
Salt filter uses whole-season tolerance, never seedling	Seedling tolerance overstates what a crop survives over a season.

Decision	Why
figures	
No AI inside the engine	Card text is fixed templates plus computed values, so nothing in the output is generated or guessed.
No network inside the engine	The demo runs with Wi-Fi off. Only `scripts/` touch the internet.

What we did, how, and why

Step 1: hour-1 checks and the NDVI request

What and how. We checked in satellite imagery that both demo points sit on cropland: an open paddy block at Anwara (Chattogram coast, salinity) and a terraced paddy block at Godagari (Barind, drought). We read the MODIS NDVI layer names and quality meanings from AppEEARS, and submitted one point request for both sites, 2000 to today.

Why. AppEEARS requests can wait in a queue for hours, so it went in first and we kept working.

Step 2: repo and the first POWER call

What and how. We set up the folders and the Python package, and a `.gitignore` that keeps out the `.env` secrets and the four planning PDFs and DOCX files. Then we made one NASA POWER request per site, 1981 to today, and saved the full response with its header.

Why. The header tells us things we must never hard-code: the fill value for missing days (−999), the API version (v2.10.2) and the data sources (MERRA2, GEOSIT and others). They go into `DATA_SOURCES.md` and onto the cards.

Step 3: the JSON contract

What and how. `models.py` defines exactly what the engine sends out: field names, types and allowed values. Unknown fields are refused, and evidence tags can only be Observed, Derived or Scenario. Every response says `schema_version: "1.0"`. The schemas are exported to `schemas/`.

Why. The frontend breaks when a field is renamed, not when a number changes. Freezing the names early lets teammates build in parallel.

Step 4: fetch POWER for both sites

What and how. `scripts/fetch_power.py` uses a disk cache, retries with a timeout, saves the raw JSON next to a clean CSV, and turns −999 into empty values. It also fetches GPM IMERG rain (as served by POWER) as a separate request.

Why. POWER may block a client that asks for the same place again and again, and the demo has to run with Wi-Fi off. So we fetch once and keep the files.

Step 5: seasons and features (the data break)

What and how. The window is the 60 days that end at the last day with data before the decision date. We use the same calendar window in every past year, and name each season by its Boro harvest year. Each season becomes one row: rain (on a log scale), mean minimum temperature and mean humidity.

Why. This step found a real problem. POWER's rain, humidity and maximum temperature jump in 1997 at both sites, and Anwara's rain is far too high from 2015 on. IMERG rain looks right but stops in mid-September 2026, so it can't cover this year's window. We decided: look-alike years come only from 1999 on, and Anwara matches on temperature and humidity, without rain. Both choices show as warnings on every card set.

Step 6: look-alike years

What and how. `analogs.py` removes the temperature trend and puts every feature on the same scale, using the training years only. It measures how far each past year is from this year (Mahalanobis distance with a Ledoit-Wolf covariance), keeps the 5 nearest, weights the closer ones more, and flags years that aren't really close. A plain Euclidean distance runs alongside, and we report how many years both methods agree on. There are two pools: all weather years from 1999, and the years with greenness data, 2001–2024.

Why. Training years only means no leakage. Ledoit-Wolf keeps the maths stable with few years. If no year is close, the card says "no close look-alike year" instead of pretending.

Step 7: the crop rule table

What and how. We drafted `rules/rules.json` ourselves instead of waiting for the science translator's file. It holds 19 crops, 28 cropping patterns and the three seasons (Rabi, Kharif-1, Kharif-2), from 5 sources. Every number carries the exact quote it came from, and anything we couldn't find is `null`.

Why. We didn't want to wait, but we also didn't want invented crop facts. The engine refuses to load a number without a quote. The file is marked unreviewed, and every card set says so. rules_json_guide.pdf explains the file in full.

Step 8: confidence (built before step 7, by agreement)

What and how. We cleaned the MODIS greenness (NDVI) data: keep good-quality points, place each on the day it was actually taken, and take each year's mean greenness in the crop's peak window, minus its trend. Each of the 5 look-alike years then votes: greener or less green than normal. `confidence.py` reports the count and the chance rate.

Why. "4 of 5 agree" sounds strong, but 5 coin tosses do that 38% of the time, and the sentence on the card says so. Godagari's dry-season greenness peaks in February, not March–April, so it uses a February window, marked for the translator to confirm.

Step 9: the cards

What and how. We wrote the card rule into `docs/ENGINE.md` before coding it. Card 1 is the current rotation. The other patterns are filtered by whole-season salt tolerance and scored against the farmer's priorities; then the top option with a legume and the top one without are picked. At 8 dS/m or more, card 3 becomes "Fallow, short oilseed, or local review".

Why. Words like "nitrogen-fixing" or "lower-water" only appear when a sourced number backs them. Unknown scores show as `null`, not zero. "Best" and "you should" never appear, and a test checks this.

Step 10: tests, fixtures and the offline check

What and how. We added the automatic tests and ran every command with the network cut off. `scripts/build_fixtures.py` writes ready-made card sets for the demo, and `docs/INTEGRATION.md`

got its first draft.

Why. If the backend fails on demo day, the frontend can still show the fixtures. Demo pair 1, "two farms, same weather": same look-alike years, different cards 2 and 3. Demo pair 2, "two regions, two hazards": Anwara (salt) against Godagari (drought).

Step 11: bug fixes and the NDVI curve

What and how. A full check of every file found and fixed ten bugs (listed at the end). The new `ndvi` command gives the greenness curve for a site: good-quality points on the day they were taken, "cloud gap" marks wherever the line must break (more than 32 days without a point), "low-vegetation window" marks, flags on marginal and post-2025 points, and the share of good data per month. At 8–12 dS/m, card 2 is now the option whose sourced salt figure reaches the farmer's class (salt-tolerant Boro).

Why. The demo needs the curve on screen (must-have 4). Breaking the line at gaps, and never filling them, keeps the chart honest about the monsoon cloud.

Step 12: the freeze gate

What and how. We checked the four must-haves: look-alike years with honest confidence; three cards with the current rotation first; priorities that change the ranking; the NDVI curve. We also checked that no card says "best" and that the disclaimer is present. All passed.

Why. This is the stop before the 240-second video: no stretch work until the basics are safe. The `demo-v1` tag is still open, because a tag needs a commit and Sudipta makes all commits.

Step 13: full NDVI

What and how. A Savitzky-Golay smoothed line now sits beside the raw points. It is fitted only inside gap-free stretches and is never drawn across a gap. Low-vegetation windows use the smoothed line. The new `ndvi-years` command gives one Boro season (1 Nov–31 May) per harvest year, so the UI can show what the greenness did in each look-alike year.

Why. The raw 16-day points are noisy. The smoothed line helps the eye, and keeping the raw points next to it means nothing is hidden.

Step 14: the backtest

What and how. Before running anything, we wrote the verdict rule into `docs/ENGINE.md`: "SS > 0 and permutation $p < 0.10$ = some skill. Otherwise: no demonstrable skill, and the analog years are shown as historical examples only." Then `backtest.py` pretended each Boro year 2001–2024 was unknown, refitted everything without it, and checked whether its look-alike years would have predicted its peak greenness better than the plain average.

Result: no demonstrable skill at either site. The skill score is -0.148 at Anwara and -0.311 at Godagari, with permutation $p = 0.53$ and 0.87. Higher agreement among the look-alike years did not go with more hits.

What we did with it. The verdict is now a warning on every analog result and card set, and a Derived evidence item on every card. The confidence sentence was reworded to describe past examples: "In 4 of the 5 past seasons that started like this one, … These are past examples, not a prediction for this season."

Why. This is the most important honesty check in the project. A judge can ask "does it work?", and we can answer with a pre-registered test, not a demo that happens to look good. Showing a negative result openly is stronger than hiding it.

Step 15: stretch items (all five)

Each one is opt-in, so the default output is unchanged.

- **What-if slider:** change this season's rain, temperature or humidity and see which past seasons look like the changed weather. Every output is tagged Scenario.
- **Tercile scenario:** look-alike years only from past seasons that turned out dry, normal or wet (ranked by IMERG rain after the decision day). Also tagged Scenario.
- **Soil wetness:** POWER's root-zone soil wetness as evidence labelled "model output". It is shown at Godagari only, because at Anwara it carries the same 2015 jump as the rain. It is never matched on.
- **4th card:** `extra_card: true` adds the next-ranked option whose dry-season crops differ from every card shown.
- **More NDVI:** Aqua satellite points beside Terra, the spread across a 3×3-pixel block, and a separate 30 m HLS curve. All of these are display only: matching, confidence and the backtest stay Terra-only.

Why. These make the demo richer without touching the tested core. Keeping them display-only means the backtest still describes exactly the method that makes the cards.

Step 16: final backtest

What and how. We reran the backtest on the cached data. It is byte-identical to the first run at both sites, and the numbers are in `docs/ENGINE.md`.

Open: the `engine-freeze` tag (Sudipta commits and tags). A POWER refetch on event day means rerunning this step and rebuilding the fixtures.

Step 17: the engine doc and the integration guide

What and how. ENGINE.md covers the method, data sources with citations, the limits panel content, Use of AI and references. INTEGRATION.md covers fresh-clone install, every CLI command, the Python API, the contract, how to show each part in the UI and the rules the UI must keep. We tested it on a scratch copy of exactly the files git would commit, with no network: the tests pass, every command runs at both sites, and the card and backtest fixtures match byte for byte apart from the timestamp.

Open: a teammate following the guide on their own machine (the step's real "Done when").

How the code fits together

Folders

Folder	What's in it
`engine/src/rotate_engine/`	The engine package. Reads files only, never the internet.
`engine/tests/`	188 tests, plus a golden copy of the Anwara cards.
`scripts/`	The only code that uses the internet: fetching POWER and NDVI. Also the fixture builder.

Folder	What's in it
`data/`	`sites.json`, POWER CSVs, raw POWER responses, NDVI files (about 8 MB). The demo runs from these.
`rules/`	`rules.json`, the cited crop table.
`schemas/`	The JSON contract, exported for the frontend.
`fixtures/`	Ready-made engine output for the demo and the UI.
`docs/`	ENGINE.md (method), INTEGRATION.md (how to use it), this note, and the rules guide.
`Plan/`	The planning documents and the build checklist.

Engine files

File	Lines	Job
`config.py`	66	The fixed numbers: 5 look-alike years, 60-day window, 55-day minimum, pool start, thresholds.
`models.py`	338	The JSON contract. Every output is checked against it before it leaves the engine.
`io.py`	110	Finds the repo folder and reads sites, POWER and NDVI files. Re-reads a file when it changes.
`seasons.py`	61	Works out the 60-day window and the same window in every past year.
`features.py`	83	Turns daily weather into one row per season, plus extreme-day counts and soil wetness.
`analogs.py`	111	Finds the 5 look-alike years, with weights and the closeness flag.
`ndvi.py`	180	Cleans MODIS and HLS greenness, gives each year's peak-window anomaly, builds the curve with gaps and smoothing.
`confidence.py`	31	Counts the votes of the 5 years and works out the chance rate.
`backtest.py`	117	Leave-one-year-out test, skill scores and the permutation p-value.
`rules.py`	200	Loads `rules.json` and refuses any number without a quote. Has the salt and water helpers.
`cards.py`	292	Filters, scores and picks the rotations, and writes the cards.
`provenance.py`	41	Writes where the data came from, using the saved POWER header.
`pipeline.py`	460	Joins everything; the public functions live here.
`__main__.py`	95	The command line: `sites`, `features`, `analogs`, `cards`, `ndvi`, `ndvi-years`, `backtest`, `schemas`.

Scripts

File	Job
`fetch_power.py`	Downloads POWER weather and IMERG rain for each site, with cache and retries.
`fetch_ndvi_appeears.py`	Downloads a finished AppEEARS task (MODIS, or HLS with `hls`); needs the Earthdata login in `.env`.
`submit_ndvi_hls.py`	Submits the HLS 30 m point task to AppEEARS.

File	Job
`fetch_ndvi_ornl.py`	Downloads Aqua NDVI and the 9×9-pixel window from ORNL (no login).
`build_fixtures.py`	Runs the engine for the demo requests and saves every fixture.

How one card set is made

1. **Request in.** The CLI or a backend calls `cards_for_site` with a site, a date and the farmer's choices (current rotation, salt class, priorities, constraints). `models.py` checks them.
2. **Weather.** `io.py` reads the site's POWER file. `seasons.py` finds the 60-day window, and `features.py` makes one row per season.
3. **Look-alike years.** `analogs.py` finds the 5 closest seasons in the weather pool (1999 on) and in the NDVI pool (2001–2024).
4. **Greenness and confidence.** `ndvi.py` gives each NDVI-pool year its greenness anomaly. `confidence.py` counts the votes and adds the chance rate.
5. **Backtest.** `backtest.py` runs once per site and data state (cached), and its verdict is attached.
6. **Cards.** `rules.py` loads the crop table. `cards.py` filters by salt, scores against the priorities, picks two alternatives, and adds the cited benefits, the risks and the gaps.
7. **Out.** `provenance.py` adds the data sources, and `pipeline.py` puts it all into one `CardSet` with the warnings and the disclaimer. The result is JSON that matches `schemas/card_set.schema.json`.

How to run it

```
.venv/bin/pip install -e "engine[dev]"
.venv/bin/python -m pytest -q engine
.venv/bin/python -m rotate_engine cards --site ctg_anwara --date 2026-11-13 --input
request.json --out cards.json
.venv/bin/python scripts/build_fixtures.py
```

Why only two sites, and what more sites would take

The engine works on cached sites only: there is no "click anywhere on the map". That is deliberate. The demo runs offline, and each site needs checks that can't be automated: that the point is on cropland, where its greenness peaks, and whether its POWER series has breaks like Anwara's rain. Arbitrary locations are out of scope for the hackathon.

The data itself covers the whole country. POWER serves any point on Earth with no login, and MODIS NDVI covers all of Bangladesh through AppEEARS or ORNL. A nationwide version would mean pre-caching a grid of points, or fetching on demand in a backend. That is roadmap work.

Adding a third site today (engine owner):

1. Add it to `data/sites.json`: id, name, lat, lon, hazard, `ndvi_id`, `ndvi_peak_doy` and `ndvi_peak_label`. Add `match_features` only if a feature has to be dropped.
2. Run `scripts/fetch_power.py <site_id>` and check the series for breaks, as in Step 5.
3. Submit a new AppEEARS MOD13Q1.061 point task with **all** sites in it. The engine reads a single results CSV for every site. Fetch it with `scripts/fetch_ndvi_appeears.py <task_id>`.
4. Check the site's greenness curve and pick its peak window.
5. Rerun `scripts/build_fixtures.py` and the tests.

What's still open

Owner (Sudipta) - The first commit, then the `demo-v1` and `engine-freeze` tags (Steps 12 and 16). - A `LICENSE` file (Apache-2.0 or MIT; the team decides). - On event day: refetch POWER so the window reaches the decision date (it ends 2 Nov today), rerun the backtest (Step 16), rebuild the fixtures and update the golden file. - Step 18: walk the team through both docs and run the engine once on the presenting laptop.

A teammate - Follow `docs/INTEGRATION.md` from a fresh clone on their own machine (Step 17's real "Done when"). - Run the demo on the presenting laptop with Wi-Fi off.

The science translator - Review `rules.json` and fill the most important nulls (salt tolerance for mungbean, grasspea and lentil). - Confirm the extreme-day thresholds (35 °C, 50 mm), Kharif-1 as a dry season, how the salt filter reads a class, the 8–12 dS/m card choice, and Godagari's February peak window and the crop behind it.

Known limits to keep in mind for the demo - The priority dial changes cards at Anwara once a salt class is set, but not at Godagari, where too few crops have sourced water or nitrogen numbers. Fixed presets are the fallback there. - The backtest found no skill. Present the look-alike years as historical examples, which is what every output already says.

Bugs found in the code checks (all fixed, each with a test)

- An unknown current-rotation id crashed. It now gives a clear error.
- The 2–4 dS/m salt class skipped the salt filter. It is now filtered.
- Card 3 repeated card 1 for a fallow farm at 8 dS/m or more. The note now goes on card 1 instead.
- The NDVI download script wrote a manifest the engine couldn't read. It now writes the right keys.
- The email address in the AppEEARS request file was removed, and the script now strips it.
- The POWER fetch script overwrote good files before checking new data. It now checks first.
- Decision dates with too little history crashed or gave cards from 1–3 years. They now stop clearly.
- A site missing from the NDVI file crashed. It now counts as "NDVI pending".
- Quoted values in `.env` broke the AppEEARS login. Quotes are now removed.
- A clean install failed 2 tests (missing script packages). The dev install now includes them.
- An unknown site crashed. It now returns a clear "not cached" result.
- A running backend could serve stale cards after a data update. The engine now re-reads files when they change.

Uses NASA data. Not endorsed by NASA.